

CS 59300

Application of Deep Learning ChatGPT APIs

Instructor: Dr. Zesheng Chen

Acknowledgements

■ Refer to the following:

- Introduction to Generative AI 2024 Spring
- <https://speech.ee.ntu.edu.tw/~hylee/genai/2024-spring.php>
- T81-559: Applications of Generative Artificial Intelligence
- https://github.com/jeffheaton/app_generative_ai

2

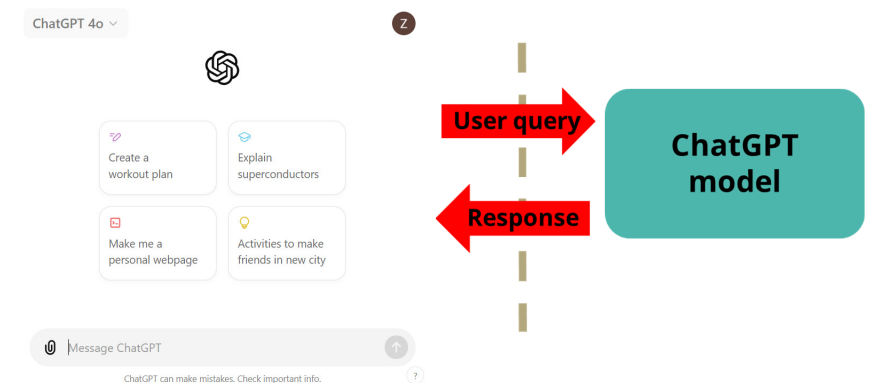
Outline

- Motivation
- Setup
- Examples

3

Motivation

■ Using ChatGPT model by browser

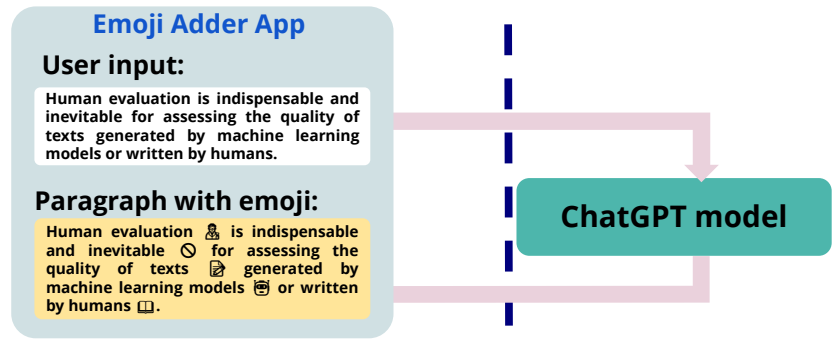


4



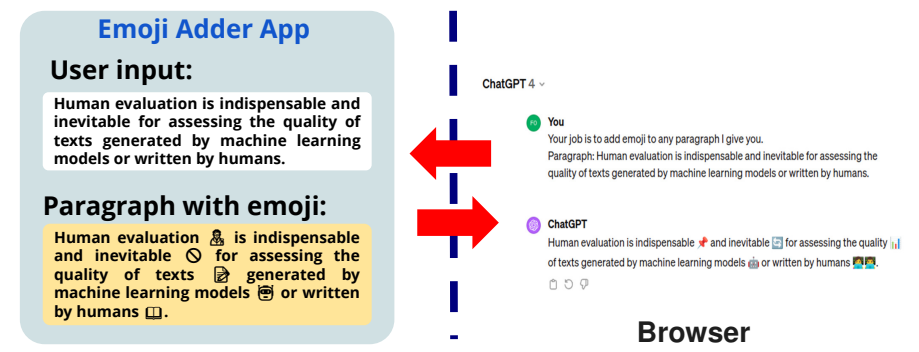
Creating Your Own Application Using ChatGPT

For example, we want to create an application that adds appropriate emojis to any given input, and you want to power it by ChatGPT.



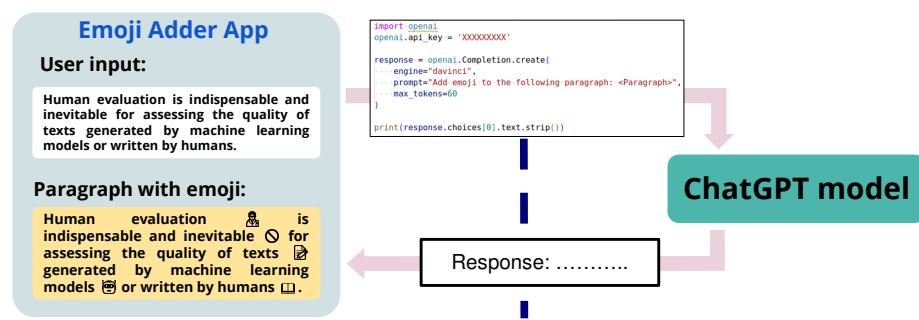
Creating Your Own Application Using ChatGPT

While you can manually copy the user input to the browser and copy the response from ChatGPT to the user end, this is too slow



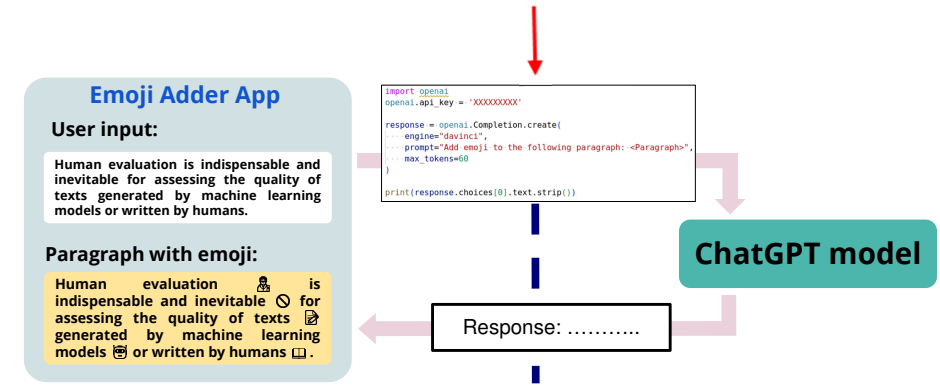
Creating Your Own Application Using ChatGPT API

The ChatGPT API (Application Programming Interface) enables programmatic interaction with the ChatGPT model



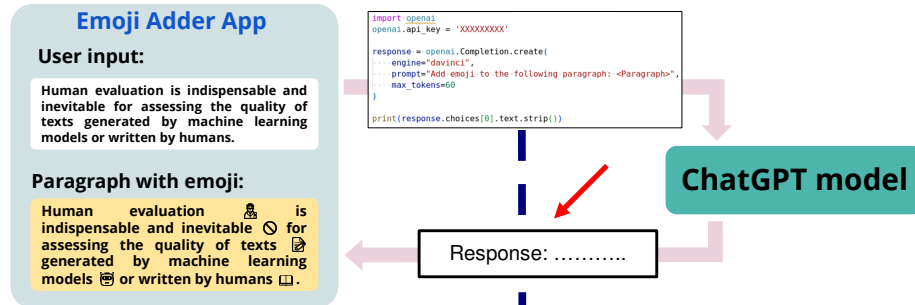
Creating Your Own Application Using ChatGPT API

You tell ChatGPT what you want it to do in a programmatic way



Creating Your Own Application Using ChatGPT API

ChatGPT returns the response by following you what you ask it to do



9

Outline

- Motivation
- **Setup**
- Examples

10

Setup

- <https://openai.com/api/>
- Sign up for an OpenAI account for API.
- Must add a payment method and some credits to your account.
 - Cannot use ChatGPT APIs without adding the credit card.
- Can start with a small amount of credits, maybe \$10, and add more as needed.
- Create an "API key".
 - Save this secret key somewhere safe and accessible.

11

Outline

- Motivation
- Setup
- **Examples**

12

Securing API Key at Google CoLab

The screenshot shows the Google Colab interface. On the left, the 'Secrets' tab is active, displaying a secret named 'OPENAI_API_KEY'. The main code cell contains the following Python code:

```
from google.colab import userdata
userdata.get('secretName')
```

Below the code cell, there is a section for 'Secret name cannot contain spaces.' and a table with columns 'Name', 'Value', and 'Actions'. The table shows the secret 'OPENAI_API_KEY' with its value masked. There is also a section for 'Access your secret keys in Python via:' with a code snippet:

13

Use Web-based ChatGPT

The screenshot shows a web-based ChatGPT interface. The prompt is: 'Please summarize the following article for me.' The article is about a community of RVers in Seattle's SoDo area. The response is a summarized version of the article. The interface includes a 'Send' button and a 'Summarization' section.

14

Web-based vs. API-based

The diagram compares web-based and API-based ChatGPT usage. On the left, a 'Browser' view shows a ChatGPT interface with a prompt and an article. On the right, an 'API Call' view shows a Python function that interacts with the ChatGPT API. The function takes a prompt and an article as input and returns a summarized response. The diagram includes labels for 'Prompt', 'Article', 'Send', and 'Summarization'.

15

Web-based vs. API-based

The diagram compares web-based and API-based ChatGPT usage. On the left, a 'Browser' view shows a ChatGPT interface with a prompt and an article. On the right, an 'API Call' view shows a Python function that interacts with the ChatGPT API. The function takes a prompt and an article as input and returns a summarized response. The diagram includes labels for 'Summarization' and 'Send'.

16



Gradio UI



```

# function to reset the conversation
def reset() -> List:
    return []

# function to generate the output,
# it will randomly generate one response from ["You are right", "haha", "I don't know"]
def interact(chatbot: List[Tuple[str, str]], user_input: str) -> List[Tuple[str, str]]:
    responses = ["You are right", "haha", "I don't know"]
    response = np.random.choice(responses, 1)[0]
    chatbot.append((user_input, response))

    return chatbot

# gradio body
with gr.Blocks() as demo:
    gr.Markdown("# Gradio Tutorial")
    chatbot = gr.Chatbot()
    input_textbox = gr.Textbox(label="Input", value = "")
    with gr.Row():
        send_button = gr.Button(value="Send")
        reset_button = gr.Button(value="Reset")
    send_button.click(interact, inputs=[chatbot, input_textbox], outputs=[chatbot])
    reset_button.click(reset, outputs=[chatbot])

demo.launch(debug = True)

```



Gradio UI

```

# function to reset the conversation
def reset() -> List:
    return []

# function to generate the output,
# it will randomly generate one response from ["You are right", "haha", "I don't know"]
def interact(chatbot: List[Tuple[str, str]], user_input: str) -> List[Tuple[str, str]]:
    responses = ["You are right", "haha", "I don't know"]
    response = np.random.choice(responses, 1)[0]
    chatbot.append((user_input, response))

    return chatbot

# gradio body
with gr.Blocks() as demo:
    gr.Markdown("# Gradio Tutorial")
    chatbot = gr.Chatbot()
    input_textbox = gr.Textbox(label="Input", value = "")
    with gr.Row():
        send_button = gr.Button(value="Send")
        reset_button = gr.Button(value="Reset")
    send_button.click(interact, inputs=[chatbot, input_textbox], outputs=[chatbot])
    reset_button.click(reset, outputs=[chatbot])

demo.launch(debug = True)

```

18



Gradio UI

When it's spinning, the gradio is connected to the colab.
When it stops, the gradio will no longer work.

```

# function to reset the conversation
def reset() -> List:
    return []

# function to generate the output,
# it will randomly generate one response from ["You are right", "haha", "I don't know"]
def interact(chatbot: List[Tuple[str, str]], user_input: str) -> List[Tuple[str, str]]:
    responses = ["You are right", "haha", "I don't know"]
    response = np.random.choice(responses, 1)[0]
    chatbot.append((user_input, response))

    return chatbot

# gradio body
with gr.Blocks() as demo:
    gr.Markdown("# Gradio Tutorial")
    chatbot = gr.Chatbot()
    input_textbox = gr.Textbox(label="Input", value = "")
    with gr.Row():
        send_button = gr.Button(value="Send")
        reset_button = gr.Button(value="Reset")
    send_button.click(interact, inputs=[chatbot, input_textbox], outputs=[chatbot])
    reset_button.click(reset, outputs=[chatbot])

demo.launch(debug = True)

```

19



Summary

- Learn how to obtain ChatGPT API key
- Learn how to call the ChatGPT API
- Learn how to design the prompts to achieve the functionality you want

20

A decorative graphic in the top-left corner of the slide. It features a solid dark blue rectangle. To its left, a staircase of squares in various shades of blue and purple extends towards the top-left corner of the slide.

Q & A

Have fun!!!